Temporal

# Build AI Agents That Survive Failure

# Nikolay Advolodkin

Staff Developer Advocate @ Temporal

Dog Dad & Roller Skater

@nikolay_a00

/in/nikolayadvolodkin/

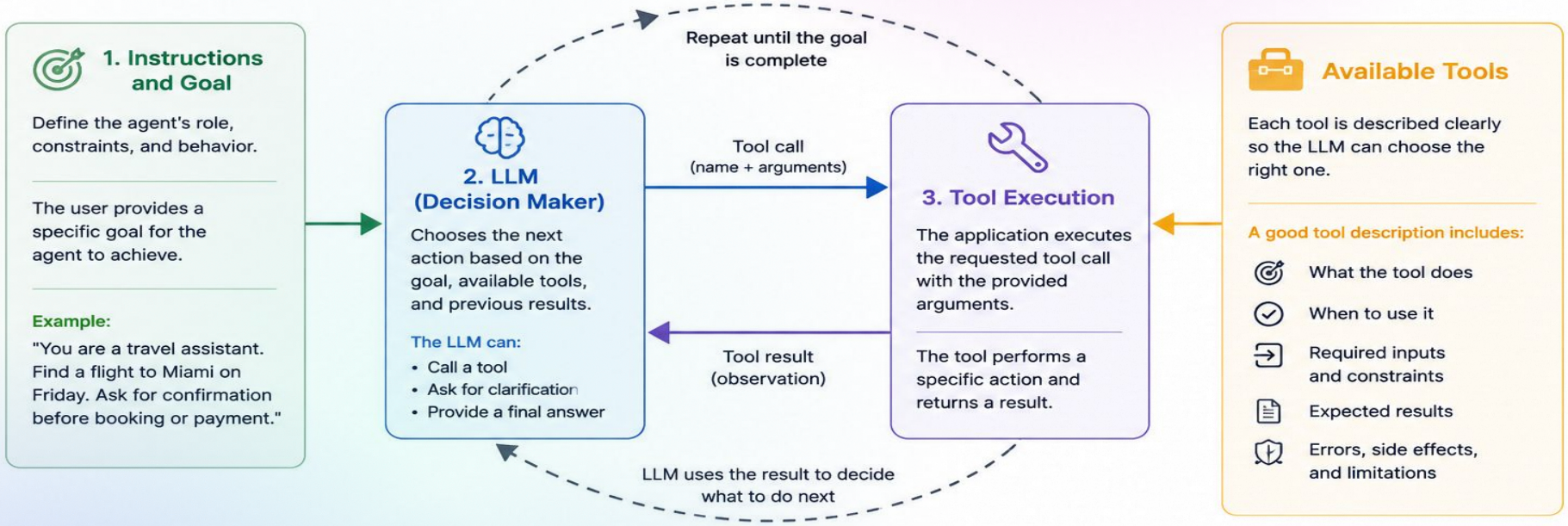
# AGENDA

- ✅ Foundations of Agentic Systems
- ✅ Building with an Agent Framework, but Durable
- ✅ Humans in the Loop (HITL)
- ✅ Orchestrating (micro)Agents

# Foundations

# The Agentic Loop

*An LLM agent observes, decides, acts, and learns—repeating until the goal is complete.*




## The Loop in 5 Steps

1. Provide the agent with instructions, goal, and available tools.
2. The LLM decides the next action (tool call, question, or final answer).
3. The application executes the tool (if called).
4. The tool result is returned to the LLM as new information.
5. The LLM decides again. Repeat until the goal is complete or a stopping condition is met.

## Key Takeaway

Clear instructions and well-described tools make the agent more reliable, safe, and effective.

# Building a **Durable** Agent

Join at:

ahaslides.com/

2TOEY

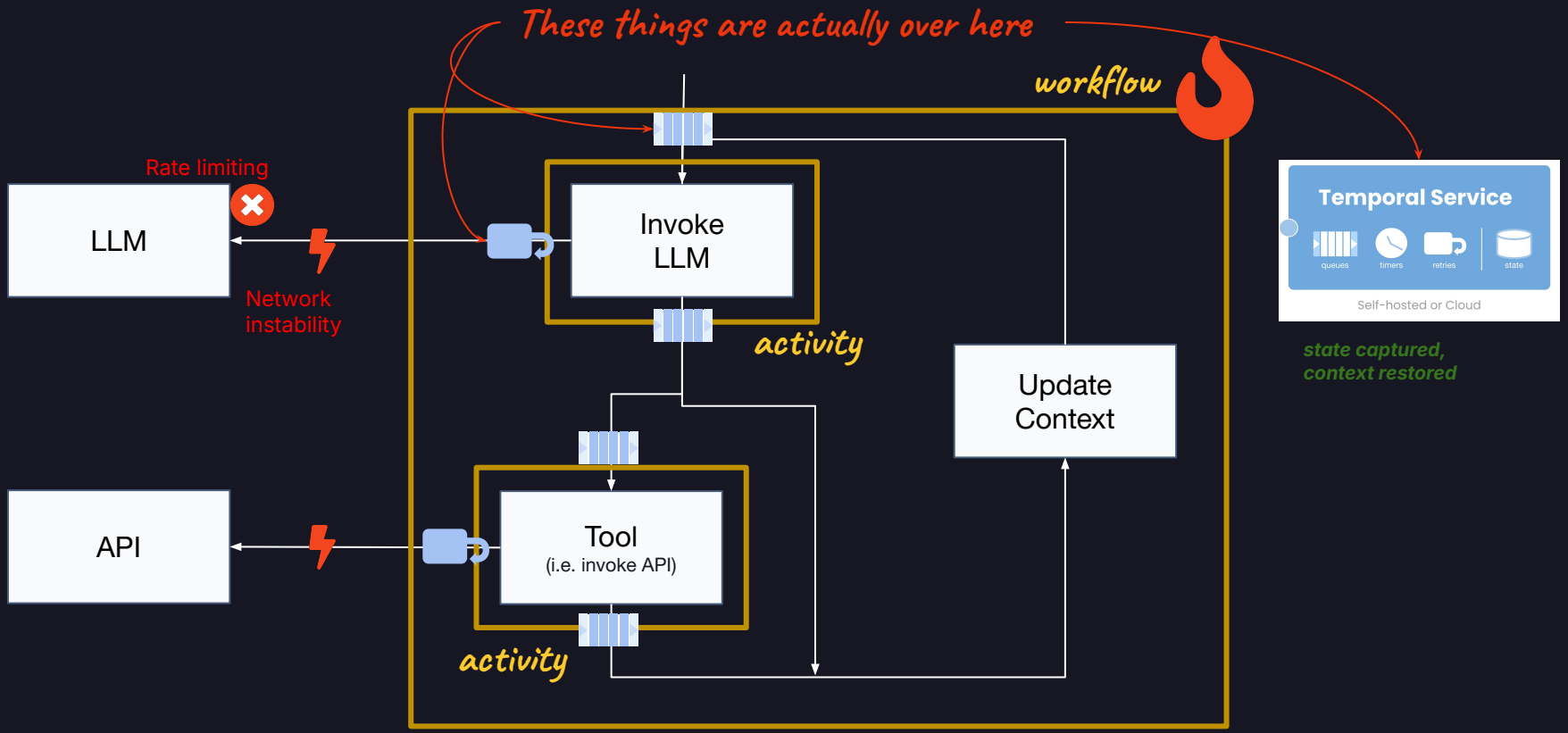
# Write code as if failure doesn’t exist

Distributed systems break, APIs fail, networks flake, and services crash. That’s not your problem anymore. Managing reliability shouldn’t mean constant firefighting.

Temporal Workflows automatically capture state at every step, and in the event of failure, can pick up exactly where they left off.

No lost progress, no orphaned processes, and no manual recovery required.

Note: The OpenAI Agents SDK + Temporal solution is currently only available for Python. Typescript coming in the future.




Agent implementation= **(recoverable)** orchestration process(es) (agentic loop)
+ LLM invocation process(es)
+ tool process(es)

***(it's a distributed system)***

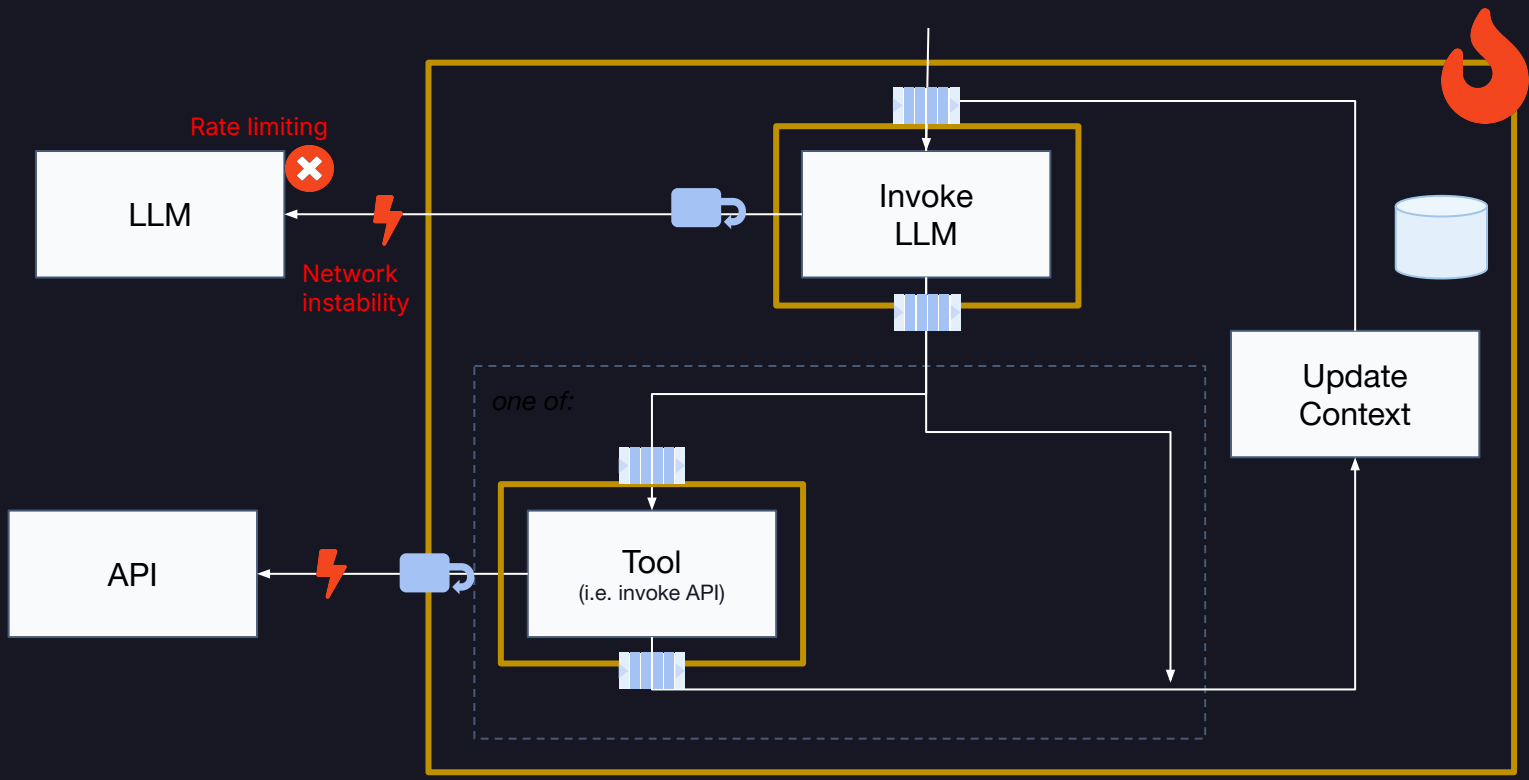
# DEMO #1

OpenAI Agents SDK

Exercises/Demos

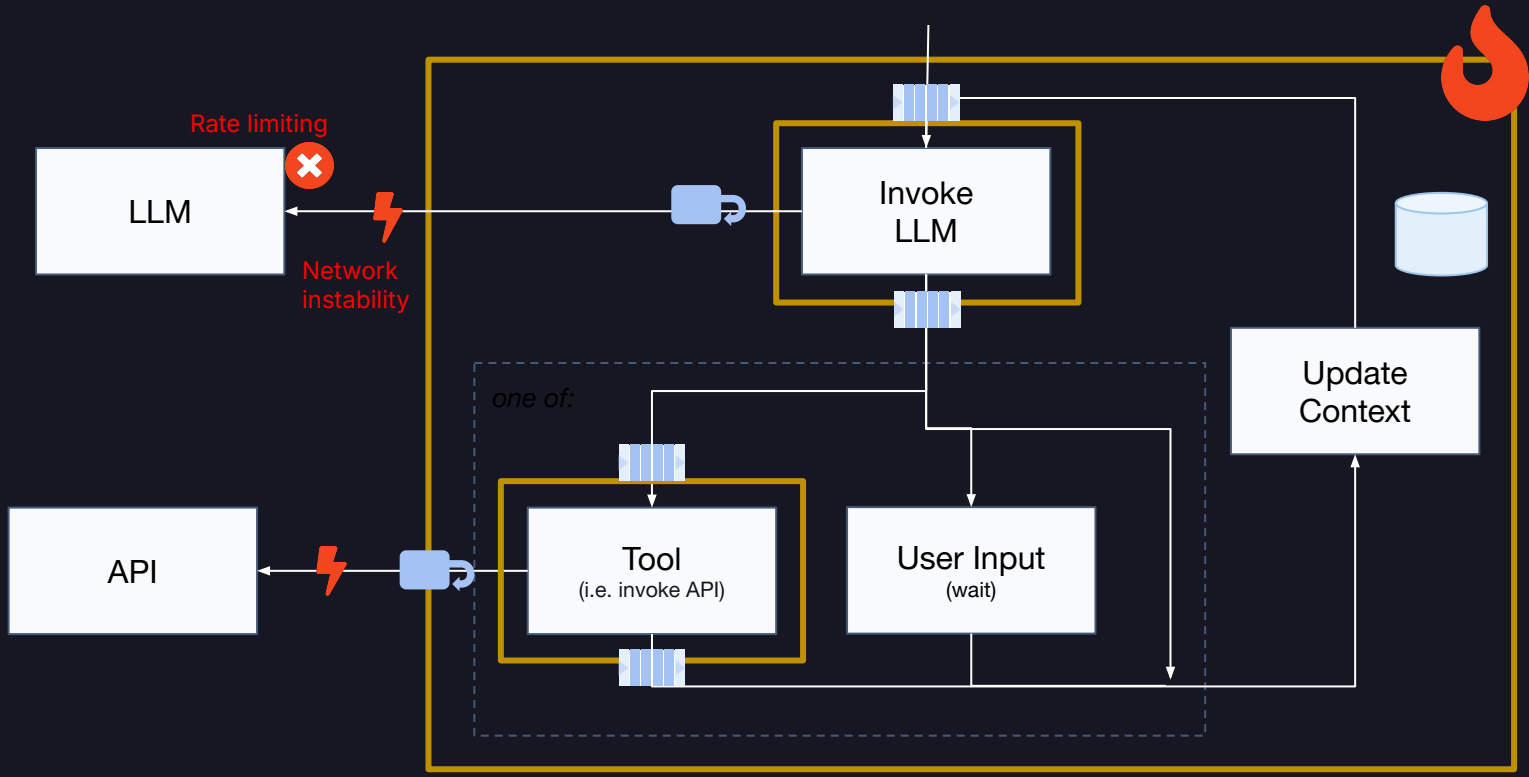
https://rb.gy/6fsbo3

# HITL




agentic loop = **(recoverable)** orchestration process(es)
+ LLM invocation process(es)
+ tool process(es)

***(it's a distributed system)***

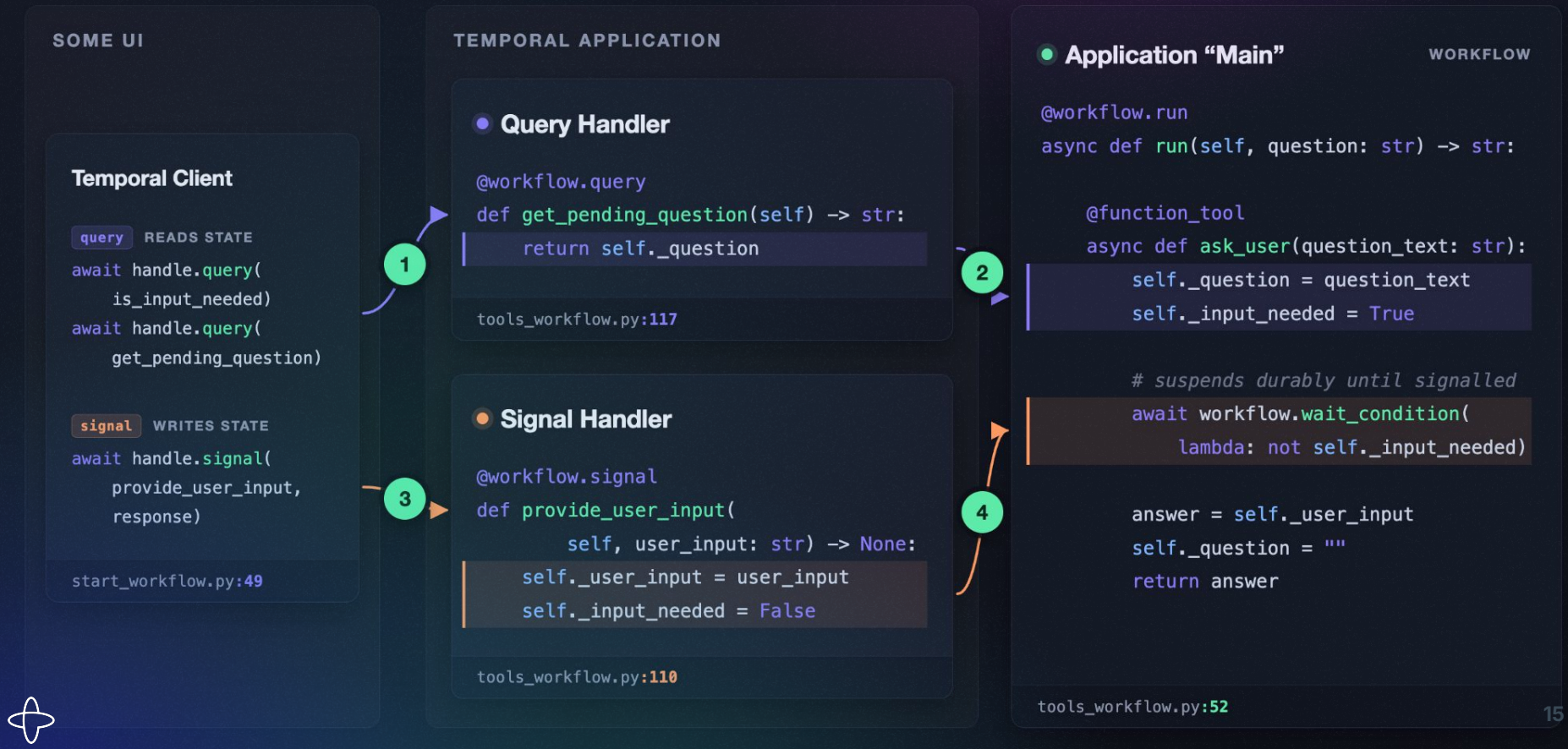



agentic loop = **(recoverable)** orchestration process(es)
+ LLM invocation process(es)
+ tool process(es)

***(it's a distributed system)***

# Overall architecture

HUMAN IN THE LOOP

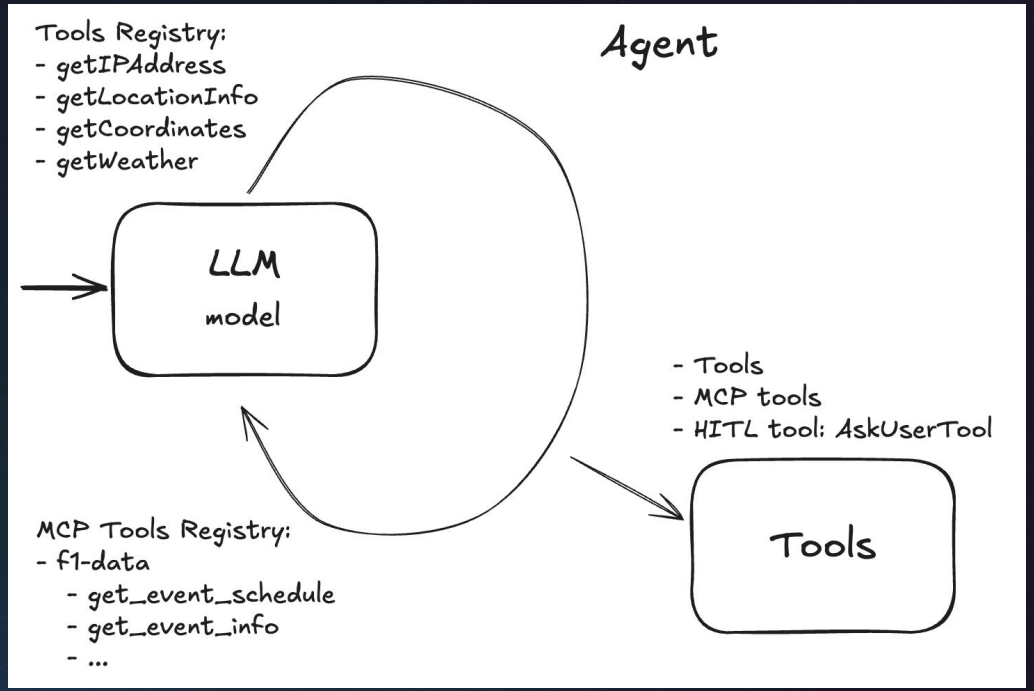


# DEMO #4

Durable **and Resource efficient** HITL

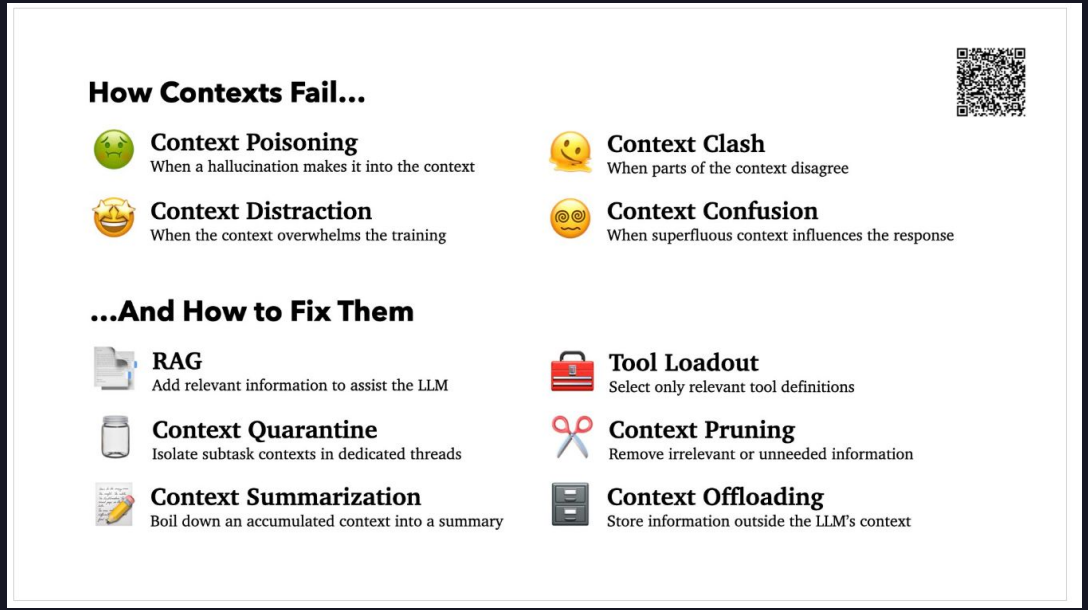
# Orchestrating (micro)Agents

# So far, we've built a single, very busy agent...



# Reasons for breaking things up

- Different teams
  - Different release cycles
  - Different tech stacks
- Modular architecture
  - Bulkheads, scale out, ...
- Scoping agents
  - Managing context
- ...




https://www.dbreunig.com/2025/07/24/why-the-term-context-engineering-matters.html

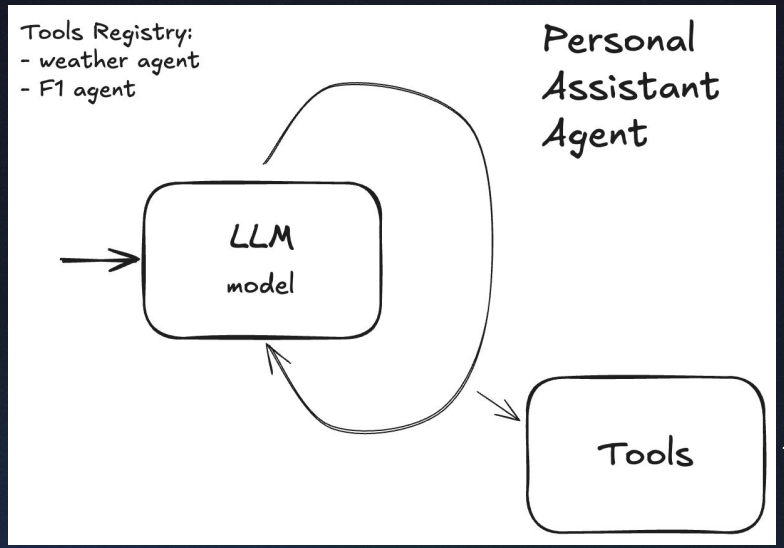
# We've seen this before!

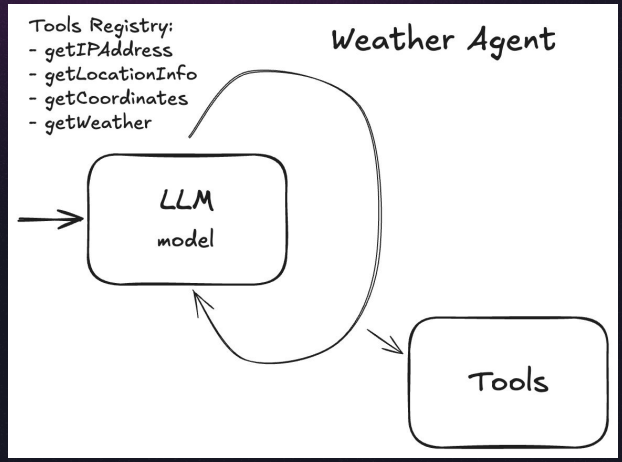
- Different teams
  - Different release cycles
  - Different tech stacks
- Modular architecture
  - Bulkheads, scale out, ...
- Scoping agents
  - Managing context
- ...

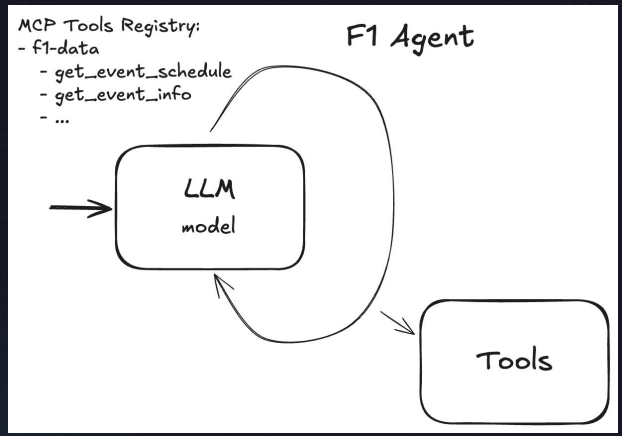
Blog

https://t.mp/micro

# So, let's break this up



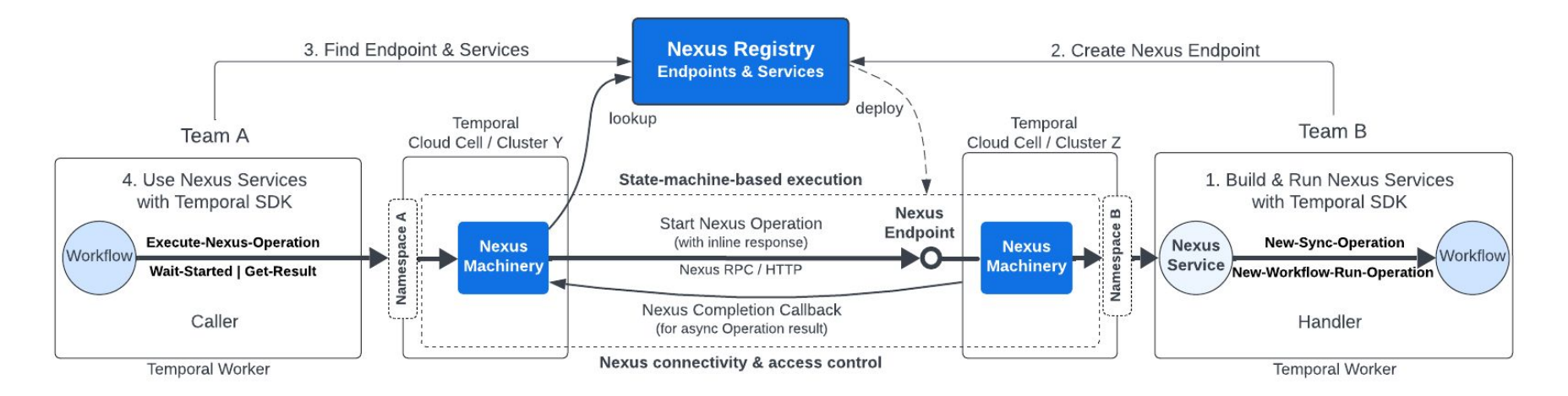



# There are options for calling sub-agents:

- Call sub-agent via activity
- Sub-agent via child workflow
- Sub-agent via Nexus

# Nexus

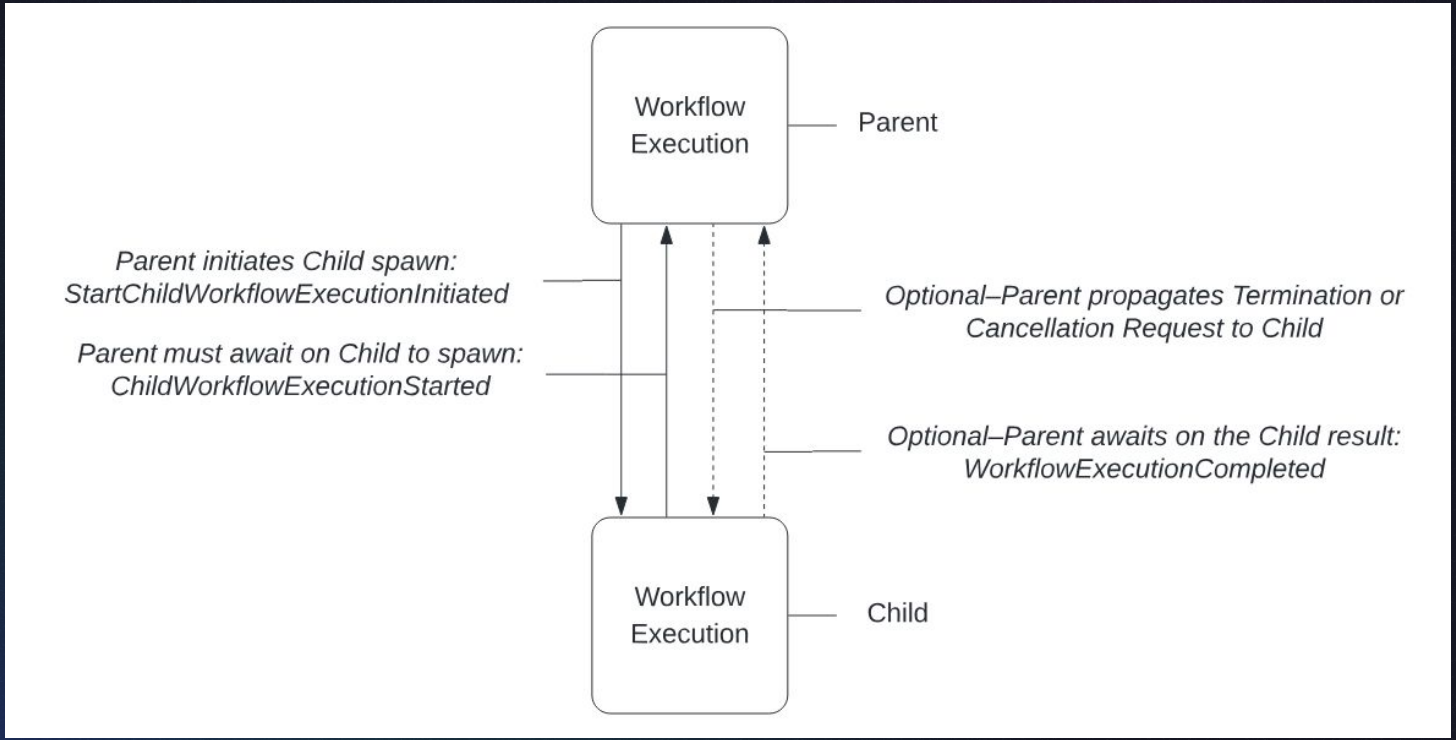



https://docs.temporal.io/nexus

# DEMO #5

Multi-agent orchestration: Nexus

# Child Workflows:




https://docs.temporal.io/child-workflows

# DEMO #5B

Multi-agent orchestration: Child Workflows

# Child workflow

Pros

- Same team, one deploy
- Direct signal + query handle
- Built-in parent-close policy
- Lower latency, no endpoint setup
- Traces propagate today

Cons

- Same namespace, same cluster
- Tight type coupling (both sides import the class)
- Sync result only
- Python-only contract

# Nexus

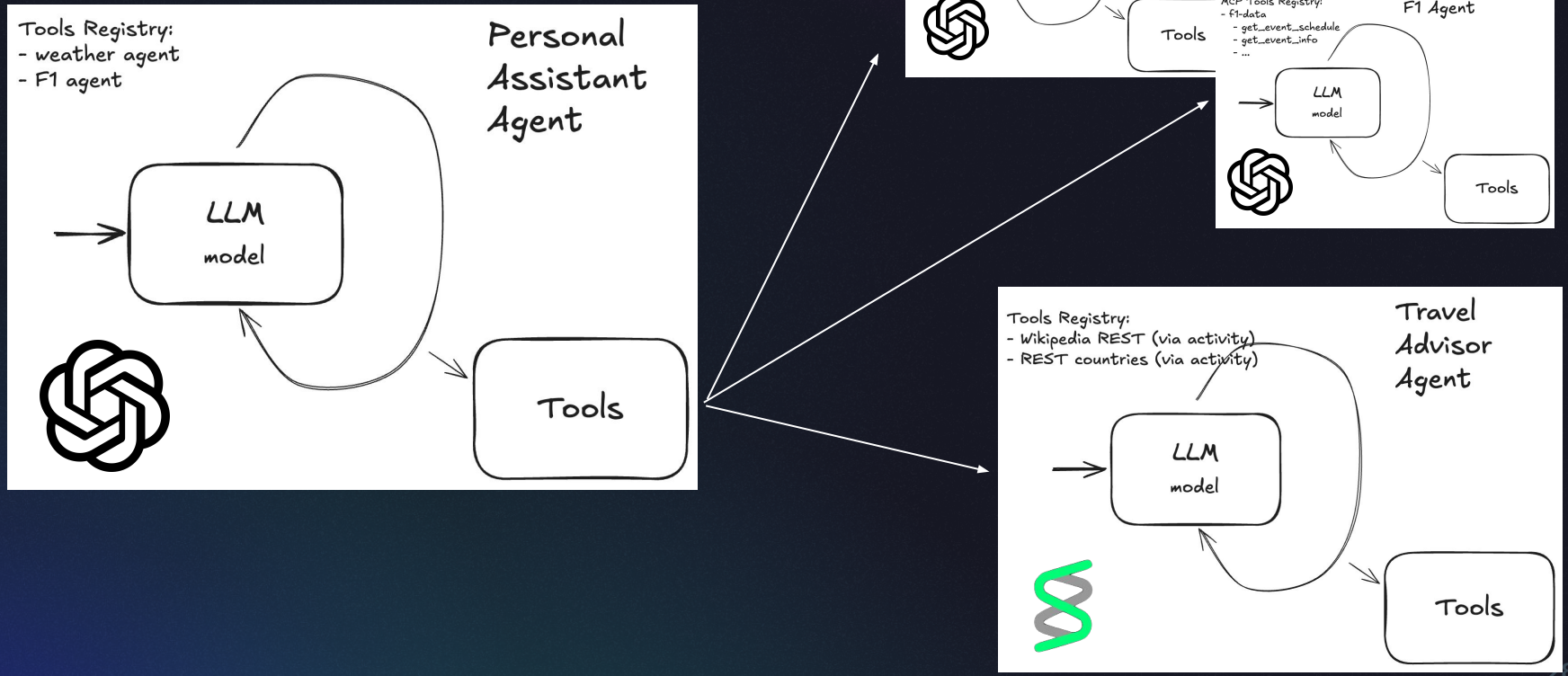
Pros

- Crosses namespaces / clusters
- Stable, language-agnostic contract
- Independent deploy cadence
- Endpoint-level policy hooks
- Async completion supported
- Signal/query support via Nexus

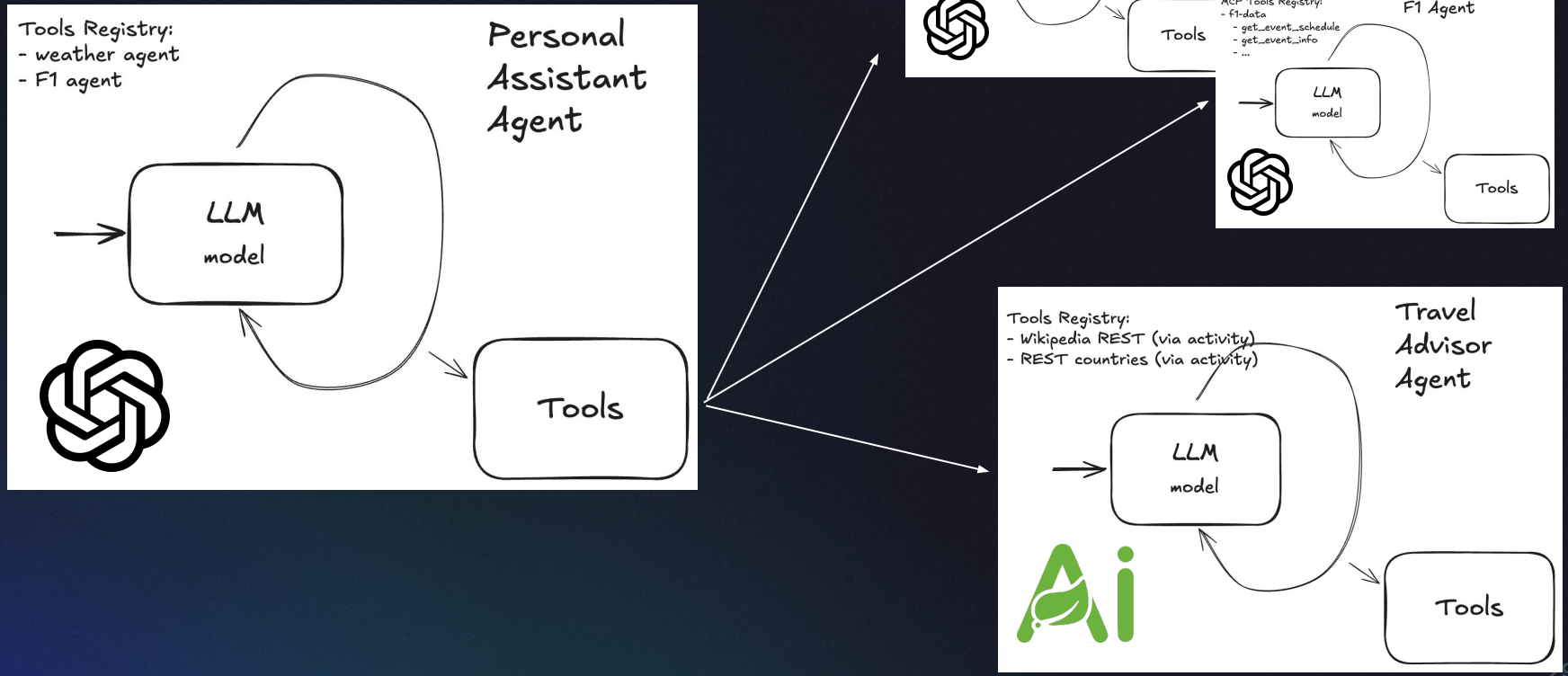
Cons

- Endpoint registration per service
- Extra hop latency
- Trace context fragmentation (current Python SDK)

# Heterogeneous Agents



# Heterogeneous Agents

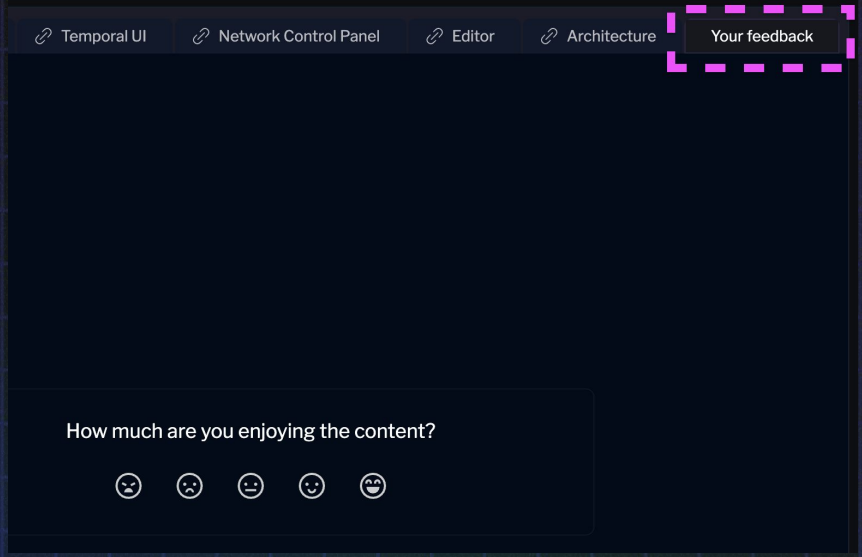


# DEMO #6

Multi-agent orchestration: Heterogeneous Agents

TBD

# THANK YOU: FEEDBACK + FOLLOW





https://rebrand.ly/link-nikk